PUNJAB TIANJIN UNIVERSITY OF TECHNOLOGY (PTUT), LAHORE • DEPT. OF SOFTWARE ENGINEERING TECHNOLOGY

BugBountyTrack

A Secure Website for Reporting and Verifying Software Security Bugs

1. Easy Bug Reporting Door
2. Automatic Browser Privacy Lock
3. Proof-Based Fix Verification

Presented By:
Taha Asadullah
Roll No: 24-ST-013 | Section: SET-B
BS Software Engineering Technology (5th Semester)

Project Supervisor & Review:
Sir Umar Hayat
Assistant Professor, PTUT Lahore
Presented to: Head of Department (HOD) & Faculty Panel

What Is BugBountyTrack? (The Simple Idea)

In plain words: BugBountyTrack is a secure web portal where independent security testers can safely report website bugs to a company, and where the company must prove the bug is truly fixed before closing the report.

ACTOR 1

Ethical Security Tester ("Bug Hunter")

- Discovers a security flaw on a company website.
- Wants a safe, legal place to report it.
- Can submit with an account or as a quick Guest.

→

OUR PLATFORM

BugBountyTrack (The Safe Bridge)

- Locks (scrambles) the report inside the browser.
- Calculates how urgent the bug is (Low to Critical).
- Tracks every step in an unchangeable history log.

→

ACTOR 2

Company Software Team ("Defenders")

- Unlocks and reads the bug report safely on screen.
- Fixes the website code and links the repair proof.
- Re-tests the fix before the ticket is allowed to close.

Core Goal: Replace risky emails and unverified "Fixed" buttons with a private, proof-based security reporting system.

Why Did I Choose This Project?

Today, small software companies and startups face three major problems when someone finds a security flaw in their website:

PROBLEM 1

Big Security Platforms Are Too Expensive

Global platforms (like HackerOne or Bugcrowd) charge thousands of dollars per month for managed services. Local startups, university teams, and small businesses simply cannot afford them.

Result: Small companies have no official door for receiving security warnings.

PROBLEM 2

Reporting Bugs Over Normal Email Is Dangerous

Without a proper portal, people send bug details through plain email, "Contact Us" forms, or social media messages. These messages sit unprotected in shared office inboxes.

Result: If an email is leaked or hacked, criminals get exact instructions to attack the website.

PROBLEM 3

Bugs Are Marked "Fixed" Without Any Real Proof

In ordinary task trackers (like Trello or Jira), a programmer can just click "Resolved" without linking any code repair or running a second check.

Result: Companies think they are safe on paper, while the security hole is still open in reality.

VALUE PROPOSITION

How BugBountyTrack Solves These Problems

WITHOUT BugBountyTrack (Current Practice)

1. No Clear Reporting Door

Testers guess who to email or post issues publicly on social media.

2. Unprotected Secret Details

Bug instructions are stored as plain readable text on emails and servers.

3. Random Priority Guessing

Teams argue over whether a bug is minor or urgent.

4. Unverified "Done" Button

Developers mark tickets "Closed" with zero proof of code repair.

WITH BugBountyTrack (Our Proposed System)

1. Official Security Policy & Guest Intake

Provides a standard security policy page and allows both Guest and Account reports.

2. Automatic Browser Privacy Lock

Locks (scrambles) the report on the user's screen before sending. Server stores only scrambled text.

3. Standardized Priority Calculator

Built-in calculator rates every bug fairly from Low to Critical (0.0 to 10.0).

4. Mandatory Code Link & Re-Test Loop

Requires a verified GitHub code update link and a passed re-test before closing.

What Are We Building? (The 3 Core Pillars)

Project Aim: Build an easy-to-use web platform that protects bug reports with browser locking and requires real proof of repair.

PILLAR 1

Public Policy & Easy Intake

Getting Reports In Safely

- Publishes a clear security rules page (security.txt) for each company.
- Registered Mode: Testers log in to track reputation & past reports.
- Guest Mode: Testers submit immediately without creating an account using a downloadable recovery key file.
- Built-in Severity Calculator rates urgency automatically.

PILLAR 2

Browser Privacy Lock

Keeping Secret Details Private

- Locks (encrypts) the bug description & steps inside the user's browser before upload.
- The central database only receives scrambled, unreadable text.
- Only the reporter and authorized company defenders can unlock it.
- Includes an Offline Company Recovery Key in case a staff member loses their laptop.

PILLAR 3

Proof-of-Fix & Re-Test Loop

Making Sure Bugs Are Truly Fixed

- Developers cannot close a bug just by clicking "Resolved".
- Must enter a GitHub code commit ID (system checks GitHub to confirm the code update exists).
- Moves ticket to "Waiting for Re-Test".
- Closes only after the tester or an internal reviewer re-tests and confirms the repair.

SYSTEM WORKFLOW

How It Works: Step-by-Step Life of a Bug Report

STEP 1

Find & Submit

By: Security Tester

Tester writes the bug report and selects severity. The browser automatically scrambles the secret text before sending.

→

STEP 2

Unlock & Review

By: Company Defender

Authorized company engineer unlocks the report on their screen, reviews the steps, and marks it "Accepted".

→

STEP 3

Fix & Link Code

By: Software Developer

Developer repairs the website code and links the GitHub commit ID. Our system checks GitHub to confirm the commit is real.

→

STEP 4

Re-Test & Verify

By: Tester / QA Auditor

Ticket enters "Waiting for Re-Test". The tester or internal reviewer tests the website again and confirms the bug is gone.

How Do Reports Close? (3 Honest, Transparent Outcomes)

1. **Verified by Researcher:** The original bug finder re-tests the fix and confirms the website is safe.
2. **Verified Internally:** An authorized team member re-tests and confirms the fix (audit log records who checked it).
3. **Unverified Timeout (If Tester Disappears):** If the reporter never replies, a manager can close it with a written reason—never faked as a verified test.

EXPECTED OUTCOME & DELIVERABLES

How Will It Come Out? (Final Prototype Output)

At the end of the project, we will deliver a complete, working web application with four main interactive screens:

SCREEN 1 — Public Policy & Bug Submission Portal

- Displays the company's official security rules and allowed domains.
- Lets testers submit a bug report with or without an account.
- Automatically locks the text in the browser and downloads a Guest Recovery File.

SCREEN 2 — Interactive Bug Priority Calculator

- Simple button-based selector where users pick how easy the bug is to exploit and what damage it causes.
- Automatically calculates an exact urgency score (0.0 to 10.0) and color badge (Low, Medium, High, Critical).

SCREEN 3 — Company Defender Triage Dashboard

- Shows all incoming bug reports sorted by urgency and status.
- Unlocks secret report details and proof-of-concept code safely inside the defender's browser.
- Includes private encrypted chat between the company and tester.

SCREEN 4 — Code Fix Checker & Audit History Log

- Checks GitHub automatically when a developer pastes a code commit ID.
- Tracks the re-test approval step.
- Generates a clean "Remediation Certificate" and permanent activity timeline.

Project Scope and Limitations

INCLUDED IN SCOPE (What We Build)

✓ **Standard Security Policy Page:** Automated security.txt and public scope page for companies.

✓ **Dual Submission Portal:** Supports both registered tester accounts and instant Guest submissions.

✓ **In-Browser Privacy Lock:** Automatic OpenPGP encryption inside the browser before upload.

✓ **Bug Priority Calculator:** Standard CVSS 3.1 severity scoring (Low, Medium, High, Critical).

✓ **Proof-of-Fix & Re-Test Engine:** GitHub commit verification and mandatory re-test workflow.

✓ **Target Platform & Users:** Modern desktop/laptop web browsers for small businesses & security testers.

LIMITATIONS & EXCLUSIONS (What It Does NOT Do)

✗ **No Automatic Website Hacking or Scanning:**
The platform does not attack websites or run automated scanners; it manages reports written by human testers.

✗ **No Financial Cash Escrow or Bank Payouts:**
BugBountyTrack is a software tracking tool, not a bank. It does not hold prize money or process cash transfers.

✗ **No Fake AI Guessing:**
We do not use unpredictable AI to judge bugs; priority scoring uses exact, standard industry rules.

✗ **No Server Password Recovery Backdoor:**
Private keys stay strictly with users. The server has no secret backdoor if both staff keys and the offline backup key are lost.

✗ **Requires Internet Access:**
Needs an active internet connection to submit reports and check GitHub commits.

EXECUTION PLAN & IMPACT

16-Week Timeline, Significance & Summary

1. 16-Week Development Schedule

Weeks 1–3

Planning & Design

Requirements, database tables & screen designs.

Weeks 4–6

Core & Privacy Lock

Login system, security.txt & browser encryption.

Weeks 7–9

Intake & Calculator

Guest/Account forms, priority calculator & chat.

Weeks 10–12

Fix & Re-Test Engine

GitHub commit check, re-test loop & audit log.

Weeks 13–16

Testing & Defense

Peer user testing, security check & final report.

2. Who Benefits from BugBountyTrack? (Significance of the Study)

Small Businesses & Startups

Get a professional, organized way to receive security alerts without paying expensive agency fees.

Security Researchers & Students

Get a safe, legal, and confidential channel to report bugs responsibly and build their portfolio.

Everyday Internet Users

Their passwords and personal data stay safer because websites must actually verify bug repairs.

Thank You! — Ready for Questions & HOD Feedback

Technology Stack: Next.js (Web UI) • PostgreSQL (Database) • OpenPGP.js (Browser Lock) • GitHub API (Commit Verification)